

Technical paper

LogicLSTM: Logically-driven long short-term memory model for fault diagnosis in gearboxes

Eduard Hoge^a, Darian M. Onchiș^a, Ruqiang Yan^{b,*}, Zheng Zhou^b

^a Faculty of Mathematics and Computer Science, Department of Computer Science, West University of Timișoara, Vasile Parvan Avenue, Nr. 4, Timișoara, 300223, Romania

^b School of Mechanical Engineering, Xi'an Jiaotong University, 28 Xianning West Road, Xi'an, 710049, China

ARTICLE INFO

Keywords:

Fault classification
Explainable artificial intelligence
Long short-term memory networks
Neuro-symbolic AI
Time-series analysis

ABSTRACT

This article introduces LogicLSTM, a hybrid neuro-symbolic model obtained by logically guiding a pretrained Long Short-Term Memory (LSTM) network with the support of a customized Logic Tensor Network (LTN). The model is further optimized by explainable AI techniques, for a refined fault classification of time-series data coming from industrial gearboxes. The framework leverages the intrinsic strengths of LSTMs deep recurrent networks for temporal data processing with logical reasoning capabilities, to improve prediction accuracy and interpretability of the classification. Our approach addresses the challenges of extracting relevant data features and integrating connectionist and symbolic methodologies to form a cohesive predictive model. Results from extensive testing show that our model significantly outperforms traditional LSTM models, particularly in complex fault scenarios where conventional methods may fail. Specifically, the hybrid model demonstrates a 16.03% average improvement in accuracy over standard LSTM models under conditions of sufficient data availability, and a 8.56% improvement in scenarios where data is scarce. This research not only demonstrates the potential of hybrid models in industrial applications but also highlights the importance of explainability in AI systems for critical decision-making processes. The proposed model's ability to interpret and explain its predictions makes it a valuable tool for advancing predictive maintenance strategies within the Industry 4.0 framework.

1. Introduction

In industrial automation, fault diagnosis and prevention are vital for operational efficiency and safety. Gearboxes are critical systems embedded in various mechanical structures, but due to prolonged operating conditions, they are prone to faults that can cause severe failures and economic losses [1,2]. Among the various types of gearboxes, planetary gearboxes are extensively used in industry due to their high torque density and compact size. However, the complexity of their design and operation makes fault diagnosis a challenging task [3–5]. This is further complicated by the nature of the data involved, which is typically time-series data captured from sensors monitoring the gearbox's operation. Such data are rich in information but require sophisticated analytical techniques to decode [6–8].

There have been vast works for gearbox fault diagnosis and can be categorized into two main groups, that is, physics-based and data-driven. Physics-based methods in gearbox fault diagnosis primarily use conservation laws to simulate the dynamic mechanism of physical systems, like lumped-parameter models [9]. Despite these models

can provide valuable insights to understand the evolution of physical damage, physics-based methods require extensive prior knowledge and usually vary significantly between different scenarios. By comparison, data-driven methods have shown considerable flexibility and prosperity in gearbox fault diagnosis. Especially, deep learning models are currently dominating the methodological approaches of fault diagnosis because they enable high accuracy for predictive maintenance, allowing early failure warning. [10–14]. Yao et al. [15] utilized the acoustic signal for gear fault diagnosis in noise condition and proposed a recursive self-attention learning method for anti-noise diagnosis. Zhou et al. [16] concerned the gear fault diagnosis problem under data limitations and proposed a semi-supervised learning method to exploit the value of unlabeled data. Zhou and Tang [17] presented a fuzzy neural network for gear fault diagnosis which can handle fault diagnosis with unseen fault types. Lin et al. [18] integrated the signal processing method with neural networks for gear fault diagnosis, where a variational mode decomposition method was used to extract feature as the input of a probabilistic neural network. These methods provide

* Corresponding author.

E-mail addresses: eduard.hogea00@e-uvr.ro (E. Hoge), darian.onchis@e-uvr.ro (D.M. Onchiș), yanruqiang@xjtu.edu.cn (R. Yan), nanwu2@stu.xjtu.edu.cn (Z. Zhou).

<https://doi.org/10.1016/j.jmsy.2024.10.003>

Received 21 July 2024; Received in revised form 27 September 2024; Accepted 8 October 2024

Available online 6 November 2024

0278-6125/© 2024 Published by Elsevier Ltd on behalf of The Society of Manufacturing Engineers.

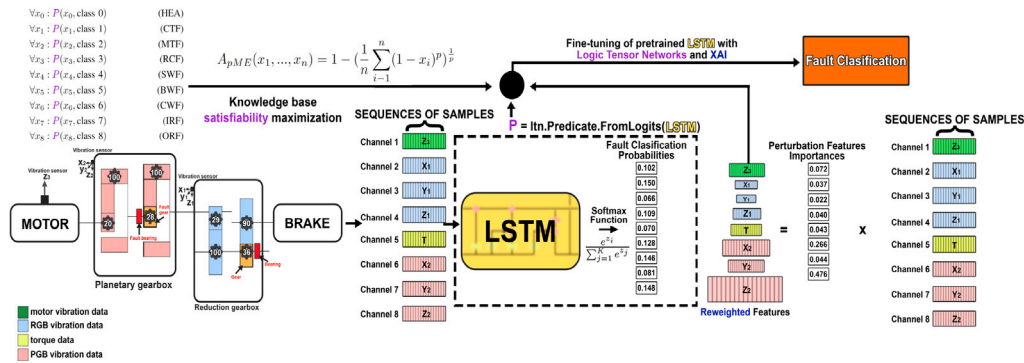


Fig. 1. System Architecture of the LogicLSTM model for gearboxes time-series fault classification. This diagram illustrates the integration of the LSTM base model with Explainable AI techniques and Logic Tensor Networks, enhancing both the predictive accuracy and the interpretability of the model.

variable approaches for gear fault diagnosis in different applications and show great success. Nevertheless, there still exists a research gap which has not been fully studied, that the lack of transparency in deep learning architectures. This can impede troubleshooting and refinement efforts, as it cannot be easily discerned why a model may be underperforming or making incorrect predictions. The issue of interpretability is especially vital in industrial settings because the inability to explain why a model flagged a particular fault can lead to hesitation in taking preventative measures, thereby nullifying the benefits of predictive maintenance.

In the field of structural health monitoring, we have previously developed a neuro-symbolic model that combines convolutional neural networks with logical reasoning in [19]. Those results demonstrated enhancements in both predictive accuracy and interpretability. Additionally, the neuro-symbolic method consistently outperformed its deep learning counterpart in terms of training time. According to our previous studies [19,20], a neuro-symbolic model incorporates interactive accuracy, allowing for real-time querying of model results, and employs deductive reasoning over data.

Such a hybrid model could be further refined by the use of Explainable Artificial Intelligence (XAI) models [21]. These models emerged as a pivotal field, elucidating the decision-making pathways of AI models—vital for deployments in critical environments where comprehension and confidence in automated decisions are imperative. Traditional machine learning models, by contrast, typically serve as black boxes, offering little to no clarity about their predictive mechanisms [22–25]. Interpretability levels differ widely; some techniques provide precise explanations of results, such as feature importance scores or decision trees, while others facilitate transparent outcome tracing, exemplified by model visualization or rule-based explanations. Notably, Vollert et al. [26] highlighted the paradox where methods designed to elucidate black box models can themselves exhibit opaque characteristics.

This paper introduces LogicLSTM, a neuro-symbolic assisted deep learning architecture, obtained by combining LSTM networks [27] and Logic Tensor Networks (LTN) [28], enhanced by XAI techniques [21]. The proposed hybrid deep learning approach, significantly improves fault classification accuracy and interpretability for time-series data coming from industrial gearboxes. This model is particularly effective with pre-existing dataset knowledge, however, as discussed in Section 4, even basic concepts can lead to significant improvements. Incorporating LTN makes the model queryable, adding flexibility and insight. This paper details the development and evaluation of the proposed model, demonstrates its superiority over conventional methods, and discusses its implications for industrial applications, thus contributing to the evolution of smart predictive maintenance strategies.

In summary, our main contribution is a logically-driven deep learning model for fault diagnosis that overcomes symbolic-neural compatibility issues and enhances fault diagnosis effectiveness. Moreover, we

introduced a novel and coherent strategy to prevent deep learning models from getting trapped in local minima, improving navigation through the model landscape and reaching optimal solutions. Our layered framework boosts each component's performance and additionally, we tackle immediate challenges in industrial fault diagnosis and broaden AI systems' applicability in industrial settings. The code for this paper is available at https://github.com/eduardhoge/DDS_Paper.

2. Related works

In the detailed study performed in [29] it is analyzed the research gap and the open challenges the neuro-symbolic field faces, stating that scalability, structure learning and applicability are issues that need to be fixed in order for the field to advance. We argue that scalability is partially solved by our proposed solution. The addition of a knowledge base does not increase the learning time of the model; on the contrary, it decreases it, as demonstrated in the previous work [19], where in order to reach a similar performance in terms of accuracy, the neuro-symbolic model needed less computation time. The incorporation of explainable methods further enhances scalability by eliminating the need to manually write all rules from axioms. While a fully dynamic system with a complex set of rules would be ideal, our implementation, which uses a customized set of rules to emphasize the learning of important features in underperforming classes, has proven effective as a working concept for a real-world application scenario.

The hybrid nature of neuro-symbolic algorithms significantly contributes to their versatility. These models have been successfully applied across a diverse range of use cases. For instance, in the medical domain, Kang et al. introduced a neuro-symbolic method designed to enhance the interpretability and generalizability of understanding free-text medical evidence in clinical trial publications [30]. In an industrial context, Gama et al. developed a neuro-symbolic explainer for rare events in predictive maintenance. Their work demonstrates how hybrid models can effectively tackle complex challenges related to anomaly detection and explanation within industrial settings [31]. The exploration of neuro-symbolic techniques demonstrates significant advances in making AI systems more interpretable and trustworthy across various domains, including but not limited to industrial automation, healthcare, and cybersecurity. These developments reflect a broader trend toward integrating domain-specific knowledge into AI, enhancing both performance and reliability of predictive models.

The general advantages and disadvantages of such models are discussed in [32]. Despite the mentioned benefits, the necessity of building a comprehensive knowledge base is highlighted as a potential burden [32]. This became the foundational idea behind the incorporation of Explainable AI in the neuro-symbolic pipeline.

The application of Explainable AI in Predictive Health Management (PHM) encompasses two predominant strategies: ad-hoc interpretable modeling and post-hoc interpretability analysis. Ad-hoc strategies incorporate intrinsic knowledge directly into AI models, improving both

interpretability and trust. Wang et al. designed a physics-informed neural network incorporating fluid mechanics into the learning process for axial piston pumps, enhancing physical interpretability [33]. Similarly, Guo et al. combined machine learning with physics-based models to estimate fuel consumption in cargo ships, reflecting the broader application of hybrid approaches in environmental sustainability [34]. Zhou et al. introduced a dynamic governing network using differential equations for time evolution analysis in lifespan prediction models [35], and Shang et al. integrated wavelet denoising in network design for fault diagnosis, embedding signal processing knowledge directly into AI models [36].

Post-hoc strategies aim to interpret models after their development. Techniques like layer-wise relevance propagation have been used to understand feature relevance in lithium-ion battery degradation [37], while SHAP values [38] have provided insights into data interpretation in bridge structural health monitoring [39].

3. Enhanced LSTM based on XAI and logic tensor networks

In this section, we introduce LogicLSTM, and we explore the technical intricacies of our hybrid model, which harnesses the power of LSTM networks, enhanced by XAI feature reweighting, and LTNs for augmented learning capabilities. The model architecture, as depicted in Fig. 1, is specifically tailored to optimize the fault classification process for planetary gearboxes using simulated time-series data.

The LSTM serves as the core of our model, handling the initial learning from time-series data to capture temporal dependencies critical for fault detection. The deep learning model is then trained on these generated sequences using cross-validation. We employ K-fold cross-validation (best results were observed with $k = 2$) for each segment, ensuring robust evaluation. Each fold is also monitored for class distribution to maintain balance. The model is compiled with a Sparse Categorical Crossentropy loss function and optimized using Adam. Learning rate adjustments and early stopping are employed to fine-tune the training process, ensuring optimal convergence without overfitting.

Upon achieving a trained LSTM model, the explainability component is introduced, focusing on permutation feature importance. This method assesses the impact of each feature on the prediction performance by randomly shuffling feature values and observing the variation in output accuracy. The calculated values for importance guide the reweighting of features, emphasizing more significant ones while diminishing the less impactful, thus refining the model's focus and enhancing its interpretability.

The integration of LTN begins once the LSTM model is pretrained. The LTN framework is employed to further enhance the model by embedding logical constraints derived from the data. These constraints are formulated as axioms within a logic-based framework, guiding the LSTM towards solutions that not only fit the data but also adhere to predefined logical rules. The training within the LTN framework is adapted to maximize the satisfiability of these axioms, using a custom loss function that promotes higher logical consistency.

The entire system is encapsulated within a TensorFlow pipeline, enabling seamless operation and integration of all components. Data flows through the LSTM, gets reweighted according to feature importance, and is finally processed by the LTN, which applies logical reasoning to fine-tune the predictions. This process not only improves the model's accuracy, but also its ability to provide explainable and logically consistent outputs.

Through detailed logging and monitoring, each step of the model's training and validation phases is recorded, allowing for comprehensive analysis and optimization. The final model is capable of distinguishing between various fault types in planetary gearboxes with high precision, demonstrating the effectiveness of combining LSTM networks with advanced XAI and LTN techniques in a cohesive hybrid model.

The proposed method works best in scenarios where there is prior knowledge about the relationship between classes or numerical values. For instance, in a time series format, knowing that a fault is likely to occur after certain values appear can significantly enhance the method's effectiveness. The data utilized for this method can come from various sources, as long as either we have prior trusted knowledge about the relationships in the data, or we can generate meaningful ones. The data does not have to be sequential, as seen in previous works [19,20,40]. Our research for this paper, however, benefited from the usage of sequences.

3.1. Explainable-AI analysis

After training and optimizing the preliminary LSTM model, we employed a post-hoc explainability approach to analyze the model's performance on validation data.

The procedures and their implementation are detailed in Algorithm 1, which outlines the systematic approach to compute and apply feature importance.

Algorithm 1 Feature Importance Assessment and Data Reweighting

```

1: Input: Trained model  $M$ , validation data  $(X_{val}, Y_{val})$ , number of iterations  $N$ , sample size  $S$ 
2: Output: Feature-weighted training and validation data
3: procedure COMPUTEFATIMP( $M, X_{val}, Y_{val}, N, S$ )
4:    $X_{subset}, Y_{subset} \leftarrow \text{SelectRandomSubset}(X_{val}, Y_{val}, S)$ 
5:    $B \leftarrow \text{CalculateBaselineAccuracies}(M, X_{subset}, Y_{subset})$ 
6:   Initialize  $I$  as array with dimensions [num_features]
7:   for  $i$  from 1 to num_features do
8:     for  $n$  from 1 to  $N$  do
9:       Shuffle  $X_{subset}[:, :, i]$ 
10:       $P \leftarrow \text{AccuraciesWithPerm}(M, X_{subset}, Y_{subset})$ 
11:      Update  $I[i]$  based on the difference  $B - P$ 
12:      Restore original order of  $X_{subset}[:, :, i]$ 
13:     end for
14:     Normalize  $I[i]$  across all iterations
15:   end for
16:   return  $I$  normalized to sum to 1
17: end procedure
18: procedure REWEIGHTFEATURES( $X, I$ )
19:   for each sample  $x$  in  $X$  do
20:     for  $j$  from 1 to num_features do
21:       Scale feature  $x[:, j]$  by  $I[j]$ 
22:     end for
23:   end for
24:   return  $X$  updated with weighted features
25: end procedure

```

Once all features have been evaluated across the designated iterations, their importances are averaged to yield a stable estimate of each feature's impact on class-specific accuracies. This averaging process results in a matrix, A , where each row corresponds to a feature and each column to a class. The matrix entries represent the average decrease in accuracy resulting from the perturbation of each feature, highlighting the critical features for accurate class classification. These findings are instrumental in guiding the reweighting of features in our dataset according to their normalized importances, thus prioritizing the most influential features.

The normalization of importances $I[i]$ for each feature across all classes is crucial for their effective utilization in data reweighting. The normalization process transforms the raw importance scores into a scale that sums to one, allowing for proportional emphasis on features based on their relative importance:

$$I_{\text{norm}}[i] = \frac{|I[i]|}{\sum_{j=1}^{\text{num_features}} |I[j]|} \quad \text{for } i = 1, \dots, \text{num_features} \quad (1)$$

Algorithm 2 Logical-guided feature-reweighting hybrid model

```

1: Input: Time-series data from gearbox sensors,  $Data$ 
2: Output: Trained and validated hybrid model,  $Model$ 
3: procedure PREPROCESSDATA( $Data$ )
4:   Standardize features to have zero mean/unit variance
5:   Segment data into training, validation, and test sets
6:   Apply sliding window technique to create sequences
7:   Balance the dataset across fault types and speeds
8:   return preprocessed data,  $Data_{prep}$ 
9: end procedure
10: procedure TRAINLSTM( $Data_{prep}$ )
11:   Initialize LSTM model,  $M_{LSTM}$  ▷ See Section 4
12:   Train  $M_{LSTM}$  using  $Data_{prep}$ 
13:   return trained LSTM model,  $M_{LSTM}$ 
14: end procedure
15: procedure TRAINLTN( $M_{LSTM}$ ,  $Data_{reweighted}$ )
16:   Integrate  $M_{LSTM}$  with LTN ▷ See Section 3.2
17:   Apply logical constraints from knowledge base
18:   Train LTN model,  $M_{LTN}$ 
19:   return trained LTN model,  $M_{LTN}$ 
20: end procedure
21: procedure EVALUATEMODEL( $M_{LTN}$ ,  $Data_{test}$ )
22:   Test  $M_{LTN}$  using different  $k$ -folds,  $Data_{test}$ 
23:   Calculate performance metrics (Accuracy, ROC-AUC)
24:   return evaluation results,  $Results$ 
25: end procedure
26:  $Data_{prep} \leftarrow$  PREPROCESSDATA( $Data$ )
27:  $M_{LSTM} \leftarrow$  TRAINLSTM( $Data_{prep}$ )
28:  $X_{val}, y_{val} \leftarrow$  ExtractValidationSet( $Data_{prep}$ )
29:  $I \leftarrow$  COMPUTEFATIMP( $M_{LSTM}$ ,  $X_{val}$ ,  $y_{val}$ ) ▷ See Section 3.1 and Algorithm 1
30:  $Data_{reweighted} \leftarrow$  REWEIGHTFEATURES( $X_{val}$ ,  $I$ )
31:  $M_{LTN} \leftarrow$  TRAINLTN( $M_{LSTM}$ ,  $Data_{reweighted}$ )
32:  $Results \leftarrow$  EVALUATEMODEL( $M_{LTN}$ ,  $Data_{test}$ )
33: display  $Results$ 

```

This ensures that the sum of the absolute values of the importances across all features equals one, maintaining a consistent scale for feature adjustment. Afterward, these importance scores are utilized to modify the dataset by weighting each feature according to its significance. The reweighting process is defined as follows:

$$X'_{i,j} = X_{i,j} \cdot I_{norm}[j] \quad \text{for each sample } i \text{ and feature } j \quad (2)$$

Here, X represents the original data matrix, and X' denotes the modified data matrix post-reweighting. This adjustment ensures that subsequent model training or predictions are significantly influenced by the features demonstrated to have a substantial impact on the model's performance, enhancing both the accuracy and robustness of the model.

3.2. LTN-based optimization

A well-defined neuro-symbolic framework that integrates deep learning with logical reasoning is represented by LTN Networks [28]. It is employed for querying, learning, and reasoning. In our experiments, LTN proved to offer effective guidance to the backbone deep learning model, helping it escape from local minima to find more optimal solutions. A critical concept within LTN is *grounding*, denoted by $\mathcal{G}$, where the semantics of traditional First-order Logic (FOL) are adapted to the real-valued tensor domain. This adaptation allows for the practical application of logical constructs to real-world data, akin to how feature extraction transforms raw data in traditional machine learning. The $\mathcal{G}$ is thus capable of mapping formulas to real numbers within the interval $[0, 1]$.

On a general view, the learning process within LTNs seeks to optimize the agreement between a computational model and a predefined logical framework known as a knowledge base ($\mathcal{K}$). The goal is to adjust the model's parameters, denoted as θ , to maximize the aggregated satisfiability of logical constraints defined in $\mathcal{K}$. Mathematically, this is formulated as:

$$\theta^* = \operatorname{argmax}_{\theta \in \Theta} \operatorname{Agg}_{\varphi \in \mathcal{K}} \mathcal{G}_{\theta}(\varphi) \quad (3)$$

where Agg is a function that combines the truth values of each logical statement in the knowledge base. This method utilizes both symbolic and sub-symbolic learning, bridging the gap between rigid logical operations and flexible neural computation.

In our case, of particular interest in the study are **predicates**, which are utilized to represent classifications, regressions, or any other metric that outputs a fuzzy degree of truth. In our system, the predicate (we will refer as P) is a wrapper applied on top of the previous LSTM model, with a softmax activation applied to its final fully connected layer. Training proceeds with the previous optimizer, Adam, retained, but the target function for minimization is adjusted. As evidenced before, the knowledge base is the cornerstone of these kinds of hybrid models. Our study shows that even by simply specifying a basic set of rules (that populate the knowledge base $\mathcal{K}$), such as the rules in Eq. (4), we can almost double the performance of the model with respect to the accuracy of the test set.

$$\text{Rules}(\forall x_i : P(x_i, \text{class } i) \quad \text{for } i = 0, 1, \dots, N) \quad (4)$$

where N represents the total number of classes.

The gradients are updated based on the satisfiability result obtained by applying (5) to the set of rules in the knowledge base. Considering that, the loss function is no longer Sparse Categorical Crossentropy; instead, it becomes $1 - A_{pME}(\text{axioms})$.

$$A_{pME}(x_1, \dots, x_n) = 1 - \left(\frac{1}{n} \sum_{i=1}^n (1 - x_i)^p \right)^{\frac{1}{p}} \quad (5)$$

The Agg is represented by A_{pME} , or Aggregated p-Mean Error, and detailed in Eq. (5), adjusts based on the individual errors x_i raised to the power p , averaged, and then transformed back by taking the p th root, effectively measuring the collective error across axioms. This measurement does not only assess the accuracy of the model but also evaluates the overall satisfiability of the axiom set, and it also displays the reasoning capabilities of one such hybrid model.

Additionally, LTN supports various logical connectives and quantifiers which are grounded using fuzzy semantics. Common fuzzy logic operators such as *not*, *and*, *or*, and *implication* are implemented using TensorFlow primitives, allowing logical operations to be directly applied within the neural network architectures. These operators play a crucial role in defining complex logical constructs that can be learned and optimized within the LTN framework.

4. Experimental evaluation

In this section, we present our evaluation of the effectiveness of the proposed method and show how it can be successfully applied to industrial data. We ran our experiments on a Linux machine. The system includes 16 GiB of system memory and an AMD EPYC-Rome Processor setup with eight CPUs.

4.1. Experimental setup

Data for the DDS dataset were collected from a drivetrain dynamics simulator (DDS) testbed, which is utilized for conducting experiments on gearbox machines under ten different operational conditions, including nine static and one dynamic condition. The control over each operational setting is achieved through adjustments in rotary speed and load. Our study specifically addresses potential faults in the planetary

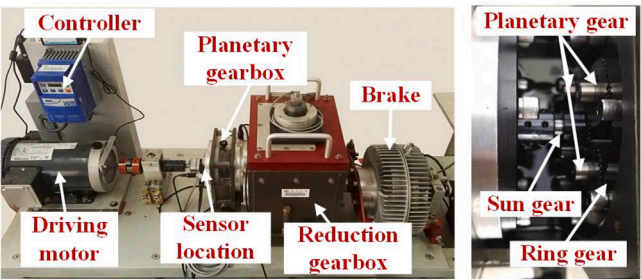


Fig. 2. Physical structure of the gearbox machine testbed used in the DDS dataset. This showcases the entire setup under different controlled conditions, highlighting the PGB and associated components.

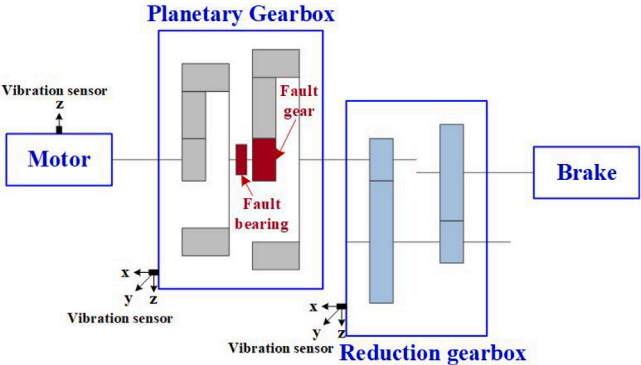


Fig. 3. Detailed illustration of the planetary gearbox and bearing positions considered in the DDS dataset. The figure highlights the specific areas of interest for the vibration data collection.

gearbox, though the vibration data encompasses all components of the testbed. The testbed configuration is illustrated in Fig. 2 and includes a controller, motor, planetary gearbox (PGB), parallel (reduction) gearbox (RGB), acceleration sensors, and a brake. The placement of the planetary gearbox and bearings is detailed in Fig. 3.

The rotational speed of the driving motor is modulated by a speed controller, capable of varying speeds from 0 to 2400 revolutions per minute (r/min). Vibration signals are captured by triaxial PCB 608A11 accelerometers mounted at the input ends of both the PGB and RGB, with a sampling frequency of 5120 Hz. Observational data also include motor vibrations and brake torque measurements.

Each operating condition is considered as a distinct domain because it exhibits a unique data distribution relative to the others. The dataset consists of 8 attributes derived from vibration data, each linked to a different sensor channel: channel 1 for motor vibrations, channels 2, 3, and 4 for y-axis, x-axis, and z-axis vibrations of the RGB, respectively; channel 5 for torque; and channels 6, 7, and 8 for y-axis, x-axis, and z-axis vibrations of the PGB (the vibration sensors are also visible in Fig. 1). Each domain is subdivided into nine classes, which include four gearbox fault classes, four bearing fault classes, and one healthy class, as outlined in Table 1 and showcased in Fig. 4.

Data description For the principal experiments, we used a targeted subset of data. Specifically, 5000 samples per fault scenario at each speed setting were chosen for the training dataset, while the testing dataset comprises 1000 samples per fault type per speed. Since gear is a rotating mechanism, our sampling strategy ensures that each sample contains at least one complete rotation cycle of the gear. The sampling frequency is 5120 Hz, and the rotation frequency ranges from 20 Hz to 50 Hz. So each rotation of gear will last from 1/20 s to 1/50 s. As each sample will have 5000 points and the duration is 5000/5120 s, one data sample contains a sufficient number of rotation cycles. Performance metrics were evaluated using the testing dataset.

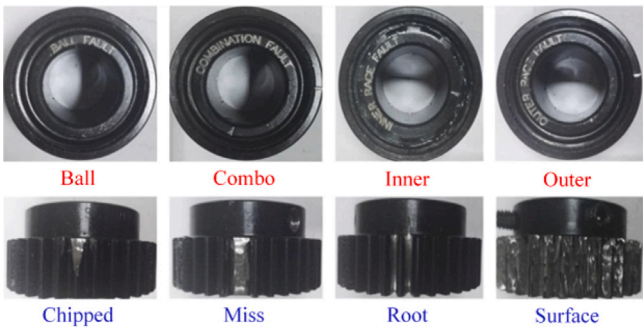


Fig. 4. Classification of Gearbox Fault Types: This images delineates the various fault types present in the planetary and reduction gearboxes, correlating each with actual physical damages from the experiments.

Table 1 Description of data classes in the DDS dataset.		
Class number	Abbreviation	Description
1	HEA	Healthy condition
2	CTF	Chipped teeth fault in Planetary Gearbox (PGB)
3	MTF	Missing teeth fault in PGB
4	RCF	Root crack fault in PGB
5	SWF	Surface wear fault in PGB
6	BWF	Ball wear fault in bearing
7	CWF	Composite wear fault in outer and inner races of bearing
8	IRF	Inner race fault in bearing
9	ORF	Outer race fault in bearing

PGB: Planetary Gearbox.

Each sample, captured at 5120 Hz, was grouped into sequences using a sliding window technique, producing overlapping subsequences. Each of these subsequences was labeled to reflect the temporal dynamics of their respective fault conditions, and each sequence received one predicted label, allowing for predictions on a per-sequence basis. If the final sequence did not have enough samples, we simply dropped it.

Data Preprocessing. Comprehensive preprocessing ensured effective training and evaluation of our machine learning model. Feature scaling, essential for LSTM, standardized features to have zero mean and unit variance. Data cleaning was conducted to ensure no missing values were present, and any detected errors in the vibration sensors were addressed by deleting those samples. Balancing was achieved by default through selecting a fixed number of samples for each fault scenario. One-hot encoding was performed on the fault types, resulting in 9 distinct classes.

For cross-validation, k-fold was employed to evaluate the model's performance more robustly. Standard practices were followed, with the dataset being divided into k equally sized folds. In each iteration, one fold was used for testing, while the remaining k – 1 folds were used for training. This process was repeated k times, with each fold serving as the test set exactly once. We experimented with 2, 4, 8, and 16 folds.

Training details. The hyperparameters of the final model were optimized using a pool of 100 samples from each class and speed, with sequences randomly selected from the dataset, ensuring they were disjoint from any data used in the segmented portions of the experimental phase. For training, we adhered to standard practices within the research community by utilizing a learning rate scheduler, dropout, L1 regularizer, and ReLU activation in the dense layers.

We also trained various other models known for their proven effectiveness in handling time-series data, their potential to capture complex temporal patterns, and their impressive results in studies addressing similar issues. Training continued until convergence, and all models were rigorously tested on the same dataset and tuned their hyperparameters on the same subset to ensure fairness. Checkpoints were implemented to monitor and save the best-performing models

throughout the training process. The Adam optimizer was consistently used across all tested models.

Comparison methods. To properly assess the complexity of the data and to provide a point of comparison to our novel method, we offer the following reference models: (1) Long Short-Term Memory, (2) Recurrent Convolutional Neural Network and (3) Synthesizer (based on Transformer architecture).

Given the complexity and temporal variability of the samples in our dataset, which focuses on predicting faults in gearbox systems, we employed a Long Short-Term Memory (LSTM) [41] deep learning architecture. This architecture serves as the backbone model for LogicLSTM. We fed sequences of predefined lengths into the model to fully leverage its ability to handle temporal dependencies, with longer sequences enhancing this capability. We used Sparse Categorical Crossentropy as the loss function, suitable for our mutually exclusive class structure. The network architecture comprises multiple LSTM layers, each followed by Dropout layers to mitigate overfitting, and Dense layers for multi-class categorization. These Dense layers employ appropriate activation functions to generate classification outputs, interpreting the features extracted by the LSTM cells and transforming them into final predictions. The model starts with an input layer, followed by several LSTM layers, each paired with a Dropout layer. Next, the sequence connects to Dense layers with ReLU as the activation function and enhanced with Dropouts. The final output layer is a Dense layer that provides the logits or probabilities for each class, based on the configuration.

The Recurrent Convolutional Neural Network (RCNN) architecture combines the strengths of convolutional neural networks (CNNs) and recurrent neural networks (RNNs) to effectively model sequential data with spatial dependencies. The network consists of a series of recurrent convolutional layers followed by a fully connected classification module.

Each layer in the RCNN is designed to capture both temporal and spatial features from the input sequences. The core component of each layer is the Recurrent Convolutional Neural Network cell (RCNNCell), which extends the traditional RNN cell by incorporating 1-dimensional convolutions. Specifically, each RCNNCell applies a convolution operation to both the input data and the hidden state from the previous time step. This allows the model to maintain and update hidden states that encapsulate information about the sequential and spatial patterns present in the data.

Formally, at each time step t , the cell takes an input $\mathbf{x}_t$ and a hidden state $\mathbf{h}_{t-1}$. The input is convolved with a set of filters to produce $\mathbf{W}_x * \mathbf{x}_t$, and the previous hidden state is convolved separately to produce $\mathbf{W}_h * \mathbf{h}_{t-1}$. The resulting feature maps are combined and passed through a non-linear activation function, $\tanh$ in our case, to produce the new hidden state $\mathbf{h}_t$. This process can be expressed mathematically as:

$$\mathbf{h}_t = \phi(\mathbf{W}_x * \mathbf{x}_t + \mathbf{W}_h * \mathbf{h}_{t-1} + \mathbf{b}),$$

where ϕ represents the non-linear activation function and $\mathbf{b}$ is the bias term.

Following the recurrent convolutional layers, a fully connected classification module is employed. This module typically includes additional convolutional layers for further feature extraction, followed by a flattening layer and one or more dense layers that map the high-dimensional feature representations to the final output classes. Swish is used as an activation function within the classification module.

The Synthesizer model, proposed by Tay et al. in [42] enhances the standard transformer architecture by integrating innovative attention mechanisms. The model consists of three primary components: an embedding layer, multiple encoder blocks, and a classification module. The embedding layer processes input sequences by segmenting them into fixed-size patches and embedding them into a higher-dimensional space, incorporating positional information to preserve sequence order.

Each encoder block contains Multi-Head Attention (MHA) and Feed Forward Network (FFN) sub-layers. The MHA employs various attention mechanisms, including the Dense Synthesizer Attention, which

synthesizes the alignment matrix directly from the input tokens without the need for pairwise dot-product calculations. The Dense Synthesizer mechanism operates as follows:

$$B_{i,h,\ell} = F_{h,\ell}(X_{i,h,\ell}) = W_{2,h,\ell}(\sigma_R(W_{1,h,\ell}(X_{i,h,\ell})))$$

where $F_{h,\ell}(\cdot)$ is a parameterized function mapping R^d to R^ℓ , and $W_{1,h,\ell}$ and $W_{2,h,\ell}$ are learnable weight matrices. To ensure fairness and develop a model well-suited for the task, we have also tested including Random Synthesizers, Factorized variants, and combinations. Outputs from the attention mechanisms are concatenated and projected back to the original hidden size. The FFN further processes these outputs using linear transformations with GELU activations, facilitating the capture of intricate patterns. The classification module aggregates features from the encoder blocks, employing either global average pooling or direct use of the class token, followed by fully connected layers to produce the final output.

Evaluation metrics. The methods have been compared and presented in the provided tables in terms of fault classification accuracy, considering the multi-class single-label task. The novel method, LogicLSTM, after proving superior, has been thoroughly tested for ROC and AUC, and a confusion matrix has been computed to offer a detailed evaluation. Additionally, knowledge base satisfiability has been an important metric studied, which is possible only with the hybrid model presented. The satisfiability of the axioms provides insight into the model's logical consistency. Computation time has also been a critical factor in evaluating how expanding the set of rules affects model complexity, and is thoroughly studied in Section 4.6.

4.2. Comparison results of fault diagnosis

The provided Table 2 presents the accuracy results for Synthesizer, RCNN, and LSTM models under various testing conditions, highlighting the performance variations across different scenarios. LogicLSTM has also been included and managed to surpass them in almost all scenarios. Focusing on the baseline methods, the analysis reveals that in the variable speed tests, LSTM consistently outperformed both Transformer and RCNN models. The Transformer's accuracy, while generally high, was slightly lower than LSTM's in these scenarios, suggesting and also backed up by the loss metrics evolution, that overfitting may be present there in the more complex methods.

Contrarily, fixed speed scenarios presented a more challenging environment, particularly for the simpler LSTM model. In most fixed speed tests, LSTM lagged behind, with notable exceptions like the 30 Hz, 4 N m scenario where it achieved the best accuracy (68.43%). The Transformer model, with a revised attention mechanism, demonstrated superior performance in these tests, achieving the highest accuracy in most fixed speed scenarios. RCNN consistently secured the second-best performance in these scenarios.

The promising results of LogicLSTM are also further studied for a thorough evaluation across all the operating scenarios. Each scenario has been individually tested, and the results have been compiled. ROC curve, AUC have been provided in Fig. 5 and a Confusion Matrix is shown in Fig. 6. The promising increase in accuracy still shows that there still is a gap to overcome in order to have a reliable model. To discern where the model lacks in performance, we provide The Receiver Operating Characteristic(ROC) curve as in Fig. 5. This is computed to assess the classifier's performance across various threshold settings for each fault category. The final model is used on the testing data. Predictions are then collected, and the true labels are binarized into a one-hot encoded matrix to facilitate multi-class classification assessment. Using a one-vs-all approach for each class, calculating the false positive rate (FPR) and true positive rate (TPR) at various threshold levels. The area under the curve (AUC) is computed to quantify the classifier's ability to distinguish between the class and all other classes. These ROC curves are then plotted with distinct colors for each class.

The confusion matrix illustrated in Fig. 6 reaffirms our findings about difficult fault scenarios. It also shows that if a class is misclassified, it would most likely be classified as inner our outer race.

Table 2

Accuracy results for Transformer, RCNN, and LSTM models. **Best results - in bold**, second best - underlined for the 3 baseline methods. LogicLSTM has also been included to prove the effectiveness of our method.

PGB state	Transformer accuracy (%)	RCNN accuracy (%)	LSTM accuracy (%)	LogicLSTM accuracy (%)
20 Hz, 0 N m	85.68	<u>82.46</u>	77.55	86.22
30 Hz, 0 N m	69.87	73.08	69.42	86.84
30 Hz, 1 N m	45.33	<u>41.32</u>	24.75	54.61
30 Hz, 2 N m	72.03	<u>69.43</u>	68.37	80.14
30 Hz, 3 N m	66.80	<u>63.89</u>	57.58	79.94
30 Hz, 4 N m	64.70	<u>65.20</u>	68.43	83.71
30 Hz, 5 N m	<u>66.33</u>	75.14	51.22	71.56
40 Hz, 0 N m	61.82	<u>61.77</u>	60.02	83.01
50 Hz, 0 N m	<u>46.23</u>	52.23	23.41	52.59
Var speed test 1	<u>96.90</u>	95.59	98.58	99.62
Var speed test 2	<u>99.31</u>	99.10	99.91	99.98
Var speed test 3	97.56	98.57	<u>98.07</u>	99.15
Var speed test 4	99.20	<u>99.54</u>	99.87	99.88
Var speed test 5	99.43	99.80	<u>99.77</u>	99.82
Var speed test 6	96.61	99.74	<u>97.66</u>	99.99
Var speed test 7	92.91	<u>97.50</u>	98.69	99.54
Var speed test 8	<u>96.30</u>	95.94	98.68	99.70
Var speed test 9	97.52	<u>97.83</u>	99.22	99.60
Var speed test 10	98.15	<u>99.73</u>	99.97	99.97

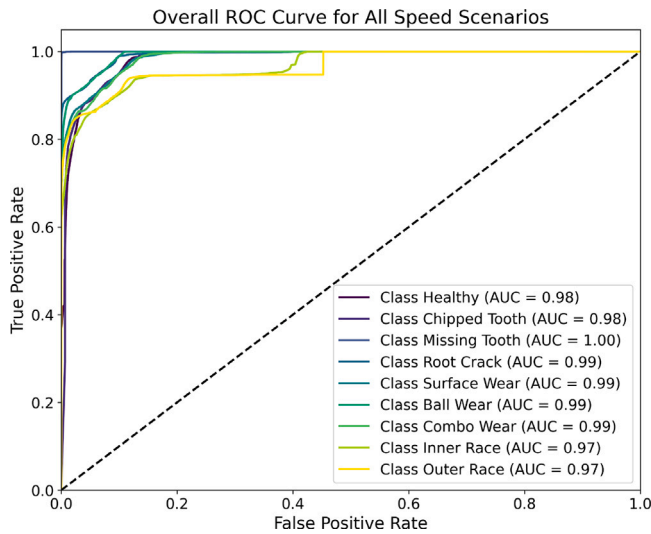


Fig. 5. Trade-off between true positive rate and false positive rate across various threshold settings for LogicLSTM. Sequences of length 20 were used on the balanced test data. The results show that even the worst performing cases have a minimum of 97% AUC.

4.3. Ablation study to evaluate efficiency of fine-tuning/convergence by LogicLSTM

To demonstrate how additional guidance can extend the capabilities of a deep learning model, we refer to Fig. 7. For this study, both the original LSTM model and its fine-tuned logic LTN counterpart were trained over a fixed span of 100 epochs. Initially, we ensured the convergence of the LSTM model by closely monitoring the validation loss and gave it an extended period to ensure no further gain in performance was possible. Upon achieving convergence, we utilized the pre-trained weights from the LSTM to initialize the LTN model, aligning it with a new objective to enhance prediction confidence.

The explainability component assesses the significance of the features in the previously trained model. This analysis generates feature importance scores, which are then used to adjust the weighting of each feature. Consequently, the pretrained model is enhanced in two primary ways: by amplifying the impact of important features and diminishing noise, and by refining the training process. The explainability part

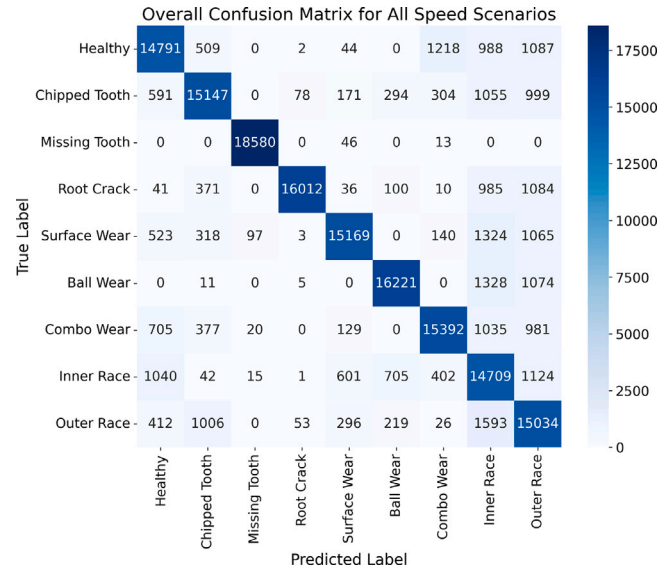


Fig. 6. Confusion matrix for fault classification using LogicLSTM. This matrix displays the number of correct and incorrect predictions for each fault type, highlighting areas of strength and opportunities for model improvement, particularly for underperforming classes.

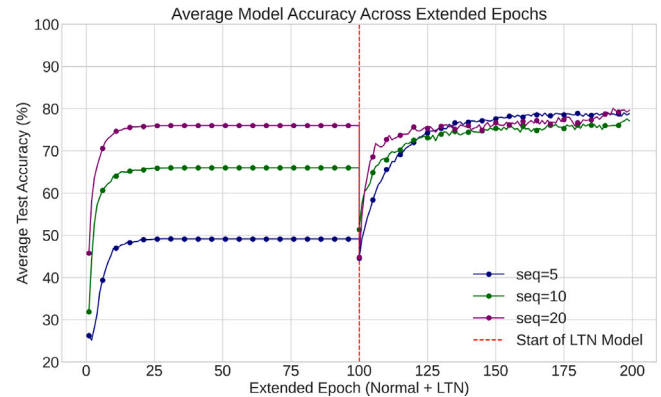


Fig. 7. Comparative validation accuracy of the LSTM and LTN models over epochs, illustrating the impact of fine-tuning on model performance. The results have been gathered as the average performance across all the tested speeds for each respective epoch. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

and the transition to the LTN model is denoted by dotted red lines at epoch 100 in Fig. 7. Notably, despite using the same optimizer and starting from a pre-trained state, the LTN's performance initially declines in its first epoch (corresponding to epoch 100 in the figure). This initial decrease is due to the reorientation of the model's focus and the reweighting of features, which temporarily destabilizes the learning dynamics.

However, the model quickly recovers, demonstrating a significant improvement in performance within the following five epochs. This swift recovery highlights the model's capability to adapt to refined learning objectives and optimized feature representations. With the new loss landscape, the model is able to escape the minima it was previously stuck in. All three test cases, with different sequence lengths, lead to a similar point, effectively increasing the learning capacity of the model.

Table 3

Comparison of LSTM and LogicLSTM model performances by speed and sequence length in detecting faults in PGBs.

PGB state	Sequence length 5		Sequence length 10		Sequence length 20		Avg. Acc. Gain (%)
	LSTM Acc. (%)	LogicLSTM Acc. (%)	LSTM Acc. (%)	LogicLSTM Acc. (%)	LSTM Acc. (%)	LogicLSTM Acc. (%)	
20 Hz, 0 N m	53.63	77.91	67.84	78.80	77.55	86.22	14.64
30 Hz, 0 N m	57.32	68.37	64.04	76.01	69.42	86.84	13.48
30 Hz, 1 N m	22.41	44.43	25.81	42.75	24.75	54.61	22.94
30 Hz, 2 N m	27.21	67.63	52.67	68.42	68.37	80.14	22.65
30 Hz, 3 N m	24.60	56.96	48.10	61.64	57.58	79.94	22.75
30 Hz, 4 N m	27.00	67.38	22.32	65.34	68.43	83.71	32.89
30 Hz, 5 N m	23.29	60.24	22.55	64.63	51.22	71.56	33.12
40 Hz, 0 N m	49.54	57.82	53.27	62.14	60.02	83.01	13.38
50 Hz, 0 N m	22.18	42.54	22.91	46.59	23.41	52.59	24.41
Var speed test 1	86.63	98.61	94.32	99.41	98.58	99.62	6.04
Var speed test 2	85.90	99.87	99.76	99.99	99.91	99.98	4.76
Var speed test 3	75.89	99.98	77.83	99.14	98.07	99.15	15.49
Var speed test 4	68.21	97.86	99.18	99.86	99.87	99.88	10.11
Var speed test 5	74.27	98.88	89.15	99.95	99.77	99.82	11.82
Var speed test 6	85.83	99.47	93.74	99.93	97.66	99.99	7.39
Var speed test 7	61.13	98.52	93.67	99.41	98.69	99.54	14.66
Var speed test 8	57.23	99.36	94.99	99.72	98.68	99.70	15.96
Var speed test 9	58.53	97.27	97.86	99.07	99.22	99.60	13.44
Var speed test 10	85.90	99.87	99.76	99.99	99.97	99.94	4.72
Average	55.09	80.68	69.46	82.25	78.48	88.20	16.03

4.4. Robustness to varying working condition and varying sequence length

To assess the efficacy and adaptability of our proposed method, we conducted a comparative analysis of model performance with and without the added XAI and LTN parts. The results, in Table 3, consistently show that the hybrid model outperforms the simpler baseline model, establishing LogicLSTM as the superior alternative. These accuracy results were derived from the best models preserved by the training checkpoints, focusing on validation accuracy, and subsequently applied to test sets across various speed categories.

To further evaluate the robustness of our method and underscore the critical role of a well-trained backbone model, we replicated the experiments using different sequence lengths while maintaining all other hyperparameters constant. Notably, under challenging conditions such as 30 Hz/4 N m and 30 Hz/5 N m, where the baseline model struggled to adequately fit the data, the refined model significantly enhanced learning capabilities, nearly tripling the accuracy.

4.5. Less training data to evaluate generalization ability of LogicLSTM

Additionally, our results underscore that the number of samples within each sequence critically impacts model performance, with reduced sample sizes leading to potential information loss and adversely affecting the model's ability to accurately predict faults. Table 4 displays a lower average gain in accuracy, but nonetheless demonstrates that even with only 10% of training data available, the guided model outperforms the standard one. The testing data was kept identical for both data availability cases. The high values indicated in the variable speed tests demonstrate an optimistic view on real-life scenarios, suggesting the potential capacity of hybrid models to address complex, industry-related problems. The superior performance of the hybrid model in these tests not only validates our methodology but also highlights its applicability in dynamic operational environments where speed and load conditions frequently vary. This adaptability is crucial for deploying deep learning solutions in industrial settings, where equipment often operates under fluctuating conditions.

4.6. Scalability and interpretability of rules in LogicLSTM

To evaluate the model's ability to respond to queries and its interpretability, we established an auxiliary set of rules. Each rule queries the model's classification accuracy with respect to the healthy state (class 0) in the 20 Hz PGB state. These queries did not impact the

training process but provided valuable insights. One example rule is: "If an instance is classified as class 0, it should not be classified as class 1", ensuring consistent and interpretable predictions. The model showed near perfect satisfiability in distinguishing class 0 from classes 1, 2, 5, 6, and 7. The levels were lower against classes 3 (0.917), 4 (0.938), and 8 (0.931), indicating areas for refinement. However, these levels differ from what we observed in the confusion matrix, which showed fewer misclassifications between class 0 and classes 3 and 4. This suggests that while the model is generally accurate, logical queries reveal underlying classification weaknesses not immediately apparent in the confusion matrix. This deeper insight underscores the value of logical queries in complementing traditional metrics, providing a comprehensive understanding of model performance.

Symbolic AI often faces challenges with scalability, especially as the dataset grows. This typically requires manual updating the rules. Our study demonstrates that using a reweighting technique allows even a general set of rules to be effective. Our method is extending the knowledge base with minimal computational overhead. We initially restricted the set of rules to consider only the performance of the healthy state and ignored the other fault scenarios. We then expanded the rules set to include all classes, increasing the complexity of the logical constraints eightfold. Despite this significant increase in the number of rules, the computation time for the LTN component only increased by 27.9%.

4.7. Counterfactual analysis of LogicLSTM

By integrating a knowledge base into our model, we have made its specific objectives explicit in terms of human logic, allowing us to investigate the relationships between features and classes. However, we are also interested in determining whether a model that maximizes the satisfiability of the aggregated set of rules in the knowledge base is also more resilient. Given the algorithmic opacity of machine learning models, understanding the minimal adjustments required to receive a different outcome provides valuable insights. In this context, counterfactual explanations serve exactly this purpose: they allow us to identify the minimal changes needed to obtain a different prediction.

Formally, and considering the time series aspect of our data, let $\mathbf{X} = [\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_T]$ be an input sequence, where $\mathbf{x}_i \in \mathbb{R}^n$ represents the feature vector at time step i , and T is the sequence length. Given a trained model f , we seek the minimal perturbation $\delta\mathbf{X} = [\delta\mathbf{x}_1, \delta\mathbf{x}_2, \dots, \delta\mathbf{x}_T]$ that satisfies:

$$\min_{\delta\mathbf{X}} \sum_{i=1}^T \|\delta\mathbf{x}_i\| \quad \text{subject to} \quad f(\mathbf{X} + \delta\mathbf{X}) \neq f(\mathbf{X}). \quad (6)$$

Table 4

Comparison of LSTM and LogicLSTM model performances by speed and sequence length in detecting faults in a Planetary Gearbox (PGB). The number of training samples used is 10% of the ones in Table 3, test samples were identical.

PGB state	Sequence length 5		Sequence length 10		Sequence length 20		Avg. Acc. gain (%)
	LSTM Acc. (%)	LogicLSTM Acc. (%)	LSTM Acc. (%)	LogicLSTM Acc. (%)	LSTM Acc. (%)	LogicLSTM Acc. (%)	
20 Hz, 0 N m	48.72	73.36	55.62	64.18	55.40	59.96	12.59
30 Hz, 0 N m	54.62	65.47	59.37	67.53	53.82	72.88	12.69
30 Hz, 1 N m	24.48	25.26	24.71	26.65	29.53	50.40	7.86
30 Hz, 2 N m	24.20	62.81	23.88	23.98	57.97	67.84	16.19
30 Hz, 3 N m	23.90	54.23	27.77	54.76	47.48	56.93	22.26
30 Hz, 4 N m	23.16	25.48	30.31	60.16	29.98	58.50	20.23
30 Hz, 5 N m	25.12	25.45	26.21	25.73	30.97	61.81	10.23
40 Hz, 0 N m	49.01	52.42	23.81	54.79	49.21	58.78	14.65
50 Hz, 0 N m	23.63	23.67	23.94	22.77	24.92	41.60	5.18
Var speed test 1	69.18	75.10	76.22	74.43	76.28	79.30	2.38
Var speed test 2	72.97	83.49	93.38	96.40	95.74	95.37	4.39
Var speed test 3	71.46	82.06	84.99	86.16	88.65	88.67	3.93
Var speed test 4	72.77	73.66	95.09	94.70	96.89	96.55	0.05
Var speed test 5	73.43	74.93	85.66	88.35	87.22	83.26	0.08
Var speed test 6	74.97	78.50	78.19	80.20	78.09	79.69	2.38
Var speed test 7	60.33	61.64	66.49	79.47	66.04	76.26	8.17
Var speed test 8	64.70	80.59	85.02	84.77	88.70	87.17	4.70
Var speed test 9	52.00	66.76	65.57	81.80	63.52	73.01	13.49
Var speed test 10	77.43	85.45	87.72	88.63	91.81	86.16	1.09
Average	51.90	61.60	58.63	66.08	63.80	72.32	8.56

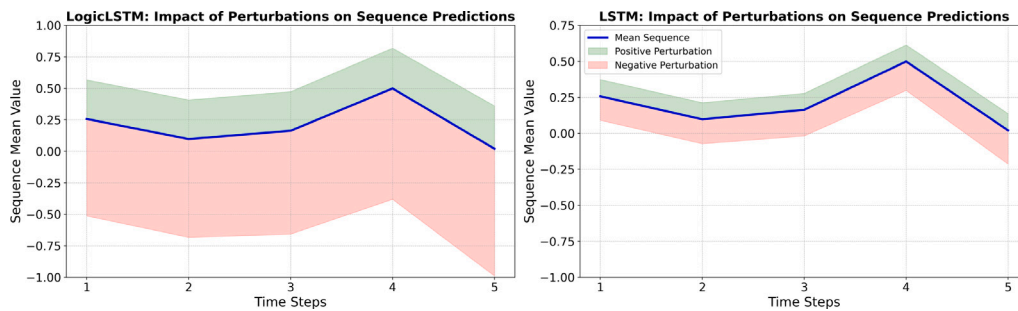


Fig. 8. Impact of perturbations on the predicted sequence values of LogicLSTM. The plot displays the mean sequence values across time steps, with shaded areas representing the positive and negative perturbations applied. Positive perturbations are shown in green, while negative perturbations are shown in red. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

As shown in Eq. (6), we aim to find the minimal perturbation δX that changes the model's output. This minimal change δX constitutes a counterfactual explanation for time series data, showing the smallest adjustments needed across all time steps to alter the model's prediction. We applied this approach to the scenario of classifying healthy vs. faulty (any fault class) conditions using our original gearbox dataset from the main experiments. We took the fully trained LSTM model and the fully trained LogicLSTM model under a variable speed scenario and conducted experiments by applying perturbations across all features at each time step, for all time steps in a fixed-length input sequence.

In addition to the feature importance used in ranking the rules, counterfactual explanations provide validation from an explainability standpoint for our new hybrid architecture. The results, shown in Fig. 8, clearly indicate that the perturbations needed to flip the model's decision are higher for the LogicLSTM model. This implies that the LogicLSTM model is more confident in its predictions. This is especially evident for positive perturbations, where only a small amount was needed to make the traditional LSTM model flip its decision.

4.8. Generalization capability

To verify the generalization capability of our proposed method, we selected two additional datasets for investigation: a gearbox fault diagnosis data from OEDI [43] which can be referred as SpectraSimulator, and the gear fault datasets from University of Connecticut (Uoc) [44].

SpectraSimulator. This dataset was recorded by using SpectraQuest's Gearbox Fault Diagnostics Simulator with 4 vibration sensors placed in four different directions. The working load of the simulator ranged from "0" to "90" percent in steps of 10 percent. Two scenario are provided: Healthy condition and Broken Tooth condition. This dataset will lead to a binary classification problem.

UoC. This dataset was collected in the University of Connecticut [44]. This dataset provides nine types of gear condition, including: healthy condition, missing tooth, root crack, spall, and chipping tip with 5 different levels of severity. The sampling frequency is 20 kHz. All the nine categories datasets were used in this investigation.

We maintained the same preprocessing and training procedures as described for the primary dataset in this paper. The backbone model architecture also remained unchanged. The only significant difference in these generalization experiments was the development of rules to accommodate the varying number of classes. Additionally, all experiments were conducted on the same machine, with both datasets being balanced to ensure consistency in the evaluation. The results in Table 5 and Fig. 9 demonstrate the strong generalization capabilities of our model across different datasets, with consistent high values for accuracies and AUCs.

5. Conclusions

This research introduced LogicLSTM, a hybrid neuro-symbolic methodology for classifying faults in time-series data derived from

Table 5
Test accuracy and AUC values for SpectraSimulator, UoC, and DDS datasets.

Dataset	Test accuracy %	HEA	CTF	MTF	RCF	SWF	BWF	CWF	IRF	ORF
SpectraSimulator	97.35	1.00	1.00	–	–	–	–	–	–	–
UoC	96.35	0.99	0.99	0.99	1.00	1.00	0.98	0.96	0.98	1.00
DDS	88.20	0.98	0.98	1.00	0.99	0.99	0.99	0.99	0.97	0.97

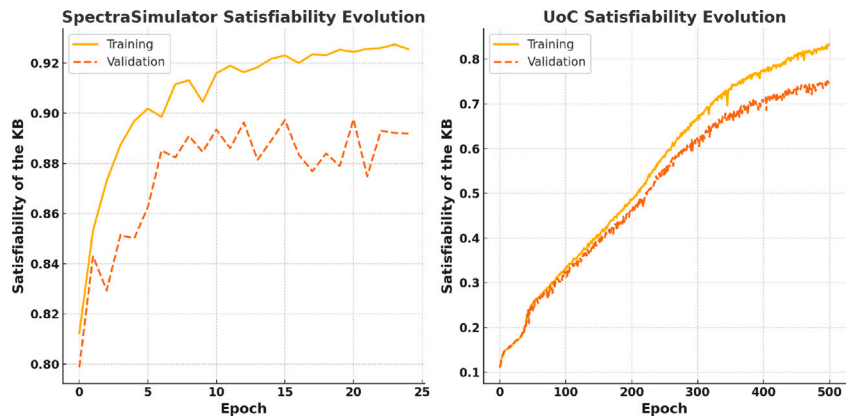


Fig. 9. Evolution of satisfiability on the SpectraSimulator and UoC datasets.

planetary gearboxes, utilizing a combined model that integrates connectionist LSTM networks with logical LTN networks, enhanced by explainable AI strategies. Our thorough evaluation has shown that this combined model markedly surpasses a standard LSTM model in accuracy, especially in intricate scenarios where the simpler model may get stuck in a local minimum.

The refinement process utilizing pretrained LSTM weights followed by retraining with the LTN model demonstrates the model’s flexibility and its improved capability to generalize from multifaceted datasets. The use of permutation feature importance further allowed the model to concentrate on pertinent features, which helped in diminishing noise and boosting predictive accuracy. Analysis of ROC curves confirms the model’s robustness, with elevated AUC values showcasing strong differentiation capabilities across various fault types.

Moreover, our experiments with Counterfactual explanations revealed that LogicLSTM is robust to data variations, further enhancing the confidence in its predictions. These explanations provide greater transparency, allowing stakeholders to better understand and trust the decision-making process. We have also conducted extensive testing using two publicly available gearbox failure prediction datasets, covering a variety of different scenarios showing how the LogicLSTM model demonstrates impressive efficiency in generalization.

CRedit authorship contribution statement

Eduard Hogea: Writing – original draft, Validation, Software, Methodology, Investigation, Data curation, Conceptualization. **Darian M. Onchis**: Writing – review & editing, Investigation, Formal analysis, Conceptualization. **Ruqiang Yan**: Writing – review & editing, Data curation. **Zheng Zhou**: Writing – review & editing, Formal analysis, Data curation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

The author Darian M. Onchis, has been partially supported by the European Union, via the oc1-2024-TES-01-19 sub-grant issued and implemented by the ENFIELD project, under the grant agreement No 101120657.

References

[1] Jing Luyang, Zhao Ming, Li Pin, Xu Xiaoqiang. A convolutional neural network based feature learning and fault diagnosis method for the condition monitoring of gearbox. *Measurement* 2017;111:1–10.

[2] Kia Shahin Hedayati, Henao Humberto, Capolino Gérard-André. Torsional vibration effects on induction machine current and torque signatures in gearbox-based electromechanical system. *IEEE Trans Ind Electron* 2009;56(11):4689–99.

[3] Jiang Weixiong, Wu Jun, Zhu Haiping, Li Xinyu, Gao Liang. Paired ensemble and group knowledge measurement for health evaluation of wind turbine gearbox under compound fault scenarios. *J Manuf Syst* 2023;70:382–94, URL <https://www.sciencedirect.com/science/article/pii/S0278612523001541>.

[4] Wang Zirui, Huang Haian, Wang Youren. Fault diagnosis of planetary gearbox using multi-criteria feature selection and heterogeneous ensemble learning classification. *Measurement* 2021;173:108654.

[5] Feng Zhipeng, Liang Ming. Complex signal analysis for planetary gearbox fault diagnosis via shift invariant dictionary learning. *Measurement* 2016;90:382–95.

[6] Pan Haiyang, Zheng Jinde, Yang Yu, Cheng Junsheng. Nonlinear sparse mode decomposition and its application in planetary gearbox fault diagnosis. *Mech Mach Theory* 2021;155:104082.

[7] Wang You-ren, Jin Qi, Sun Guo-dong, Sun Can-fei. Planetary gearbox fault feature learning using conditional variational neural networks under noise environment. *Knowl-Based Syst* 2019;163:438–49.

[8] Ravikumar KN, Yadav Akhilesh, Kumar Hemantha, Gangadharan KV, Narasimhadhan AV. Gearbox fault diagnosis based on multi-scale deep residual learning and stacked LSTM model. *Measurement* 2021;186:110099.

[9] Shen Zhixian, Qiao Baijie, Yang Laihao, Luo Wei, Yang Zhibo, Chen Xuefeng. Fault mechanism and dynamic modeling of planetary gear with gear wear. *Mech Mach Theory* 2021;155:104098.

[10] Leukel Joerg, González Julian, Riekert Martin. Adoption of machine learning technology for failure prediction in industrial maintenance: A systematic review. *J Manuf Syst* 2021;61:87–96, URL <https://www.sciencedirect.com/science/article/pii/S0278612521001849>.

[11] Paraschos Panagiotis D, Koulinas Georgios K, Koulouriotis Dimitrios E. Reinforcement learning for combined production-maintenance and quality control of a manufacturing system with deterioration failures. *J Manuf Syst* 2020;56:470–83, URL <https://www.sciencedirect.com/science/article/pii/S027861252030114X>.

[12] Quatrini Elena, Costantino Francesco, Di Gravio Giulio, Patriarca Riccardo. Machine learning for anomaly detection and process phase classification to improve safety and maintenance activities. *J Manuf Syst* 2020;56:117–32, URL <https://www.sciencedirect.com/science/article/pii/S0278612520300765>.

- [13] Zonta Tiago, da Costa Cristiano André, Zeiser Felipe A, de Oliveira Ramos Gabriel, Kunst Rafael, da Rosa Righi Rodrigo. A predictive maintenance model for optimizing production schedule using deep neural networks. *J Manuf Syst* 2022;62:450–62, URL <https://www.sciencedirect.com/science/article/pii/S0278612521002624>.
- [14] Zhai Simon, Gehring Benedikt, Reinhart Gunther. Enabling predictive maintenance integrated production scheduling by operation-specific health prognostics with generative deep learning. *J Manuf Syst* 2021;61:830–55, URL <https://www.sciencedirect.com/science/article/pii/S0278612521000340>.
- [15] Yao Yong, Gui Gui, Yang Suixian, Zhang Sen. A recursive multi-head self-attention learning for acoustic-based gear fault diagnosis in real-industrial noise condition. *Eng Appl Artif Intell* 2024;133:108240.
- [16] Zhou Kai, Diehl Edward, Tang Jiong. Deep convolutional generative adversarial network with semi-supervised learning enabled physics elucidation for extended gear fault diagnosis under data limitations. *Mech Syst Signal Process* 2023;185:109772.
- [17] Zhou Kai, Tang Jiong. Harnessing fuzzy neural network for gear fault diagnosis with limited data labels. *Int J Adv Manuf Technol* 2021;115(4):1005–19.
- [18] Lin Ying, Xiao Maohua, Liu Huijia, Li Zhuolong, Zhou Shuang, Xu Xiaomei, Wang Dicheng. Gear fault diagnosis based on CS-improved variational mode decomposition and probabilistic neural network. *Measurement* 2022;192:110913.
- [19] Onchis Darian M, Gillich Gilbert-Rainer, Hogeia Eduard, Tufisi Cristian. Neuro-symbolic model for cantilever beams damage detection. *Comput Ind* 2023;151:103991.
- [20] Onchis Darian, Istin Codruta, Hogeia Eduard. A neuro-symbolic classifier with optimized satisfiability for monitoring security alerts in network traffic. *Appl Sci* 2022;12(22):11502.
- [21] Onchis Darian M, Gillich Gilbert-Rainer. Stable and explainable deep learning damage prediction for prismatic cantilever steel beam. *Comput Ind* 2021;125:103359.
- [22] Tchuente Dieudonné, Lonlac Jerry, Kamsu-Foguem Bernard. A methodological and theoretical framework for implementing explainable artificial intelligence (XAI) in business applications. *Comput Ind* 2024;155:104044, URL <https://www.sciencedirect.com/science/article/pii/S016636152300194X>.
- [23] Suhail Sabah, Iqbal Mubashar, Hussain Rasheed, Jurdak Raja. ENIGMA: An explainable digital twin security solution for cyber-physical systems. *Comput Ind* 2023;151:103961.
- [24] Costi Flavia, Onchis Darian, Hogeia Eduard, Istin Codruta. Predictive modeling for diabetes using graphlime. 2024, p. 1–7, medRxiv 2024-2003.
- [25] Wang Jinjiang, Li Yilin, Gao Robert X, Zhang Fengli. Hybrid physics-based and data-driven models for smart manufacturing: Modelling, simulation, and explainability. *J Manuf Syst* 2022;63:381–91, URL <https://www.sciencedirect.com/science/article/pii/S0278612522000541>.
- [26] Vollert Simon, Atzmueller Martin, Theissler Andreas. Interpretable machine learning: A brief survey from the predictive maintenance perspective. In: 2021 26th IEEE international conference on emerging technologies and factory automation. ETFA, IEEE; 2021, p. 01–8.
- [27] Graves Alex, Graves Alex. Long short-term memory. In: Supervised sequence labelling with recurrent neural networks. Springer; 2012, p. 37–45.
- [28] Badreddine Samy, Garcez Artur d'Avila, Serafini Luciano, Spranger Michael. Logic tensor networks. *Artificial Intelligence* 2022;303:103649.
- [29] Hitzler Pascal, Sarker Md Kamruzzaman. Neuro-symbolic artificial intelligence: The state of the art. 2022.
- [30] Kang Tian, Turfah Ali, Kim Jaehyun, Perotte Adler, Weng Chunhua. A neuro-symbolic method for understanding free-text medical evidence. *J Am Med Inform Assoc* 2021;28(8):1703–11.
- [31] Gama João, Ribeiro Rita P, Mastelini Saulo, Davarid Narjes, Veloso Bruno. A neuro-symbolic explainer for rare events: A case study on predictive maintenance. 2024, arXiv preprint arXiv:2404.14455.
- [32] Onchis Darian M, Istin Codruta, Eduard-Florin Hogeia. Advantages of a neuro-symbolic solution for monitoring IT infrastructures alerts. In: 2022 24th international symposium on symbolic and numeric algorithms for scientific computing. SYNASC, IEEE; 2022, p. 189–94.
- [33] Wang Zhiying, Zhou Zheng, Xu Wengang, Sun Chuang, Yan Ruqiang. Physics informed neural networks for fault severity identification of axial piston pumps. *J Manuf Syst* 2023;71:421–37.
- [34] Guo Bingjie, Liang Qin, Tsvete Hans Anton, Brinks Hendrik, Vanem Erik. Combined machine learning and physics-based models for estimating fuel consumption of cargo ships. *Ocean Eng* 2022;255:111435.
- [35] Zhou Zheng, Li Tianfu, Zhao Zhibin, Sun Chuang, Chen Xuefeng, Yan Ruqiang, Jia Jide. Time-varying trajectory modeling via dynamic governing network for remaining useful life prediction. *Mech Syst Signal Process* 2023;182:109610.
- [36] Shang Zuogang, Zhao Zhibin, Yan Ruqiang. Denoising fault-aware wavelet network: A signal processing informed neural network for fault diagnosis. *Chin J Mech Eng* 2023;36(1):9.
- [37] Wang Fujin, Zhao Zhibin, Zhai Zhi, Shang Zuogang, Yan Ruqiang, Chen Xuefeng. Explainability-driven model improvement for SOH estimation of lithium-ion battery. *Reliab Eng Syst Saf* 2023;232:109046.
- [38] Lundberg Scott M, Lee Su-In. A unified approach to interpreting model predictions. In: Guyon I, Luxburg UV, Bengio S, Wallach H, Fergus R, Vishwanathan S, Garnett R, editors. Advances in neural information processing systems 30. Curran Associates, Inc.; 2017, p. 4765–74, URL <http://papers.nips.cc/paper/7062-a-unified-approach-to-interpreting-model-predictions.pdf>.
- [39] Martakis Panagiotis, Movsessian Artur, Reuland Yves, Pai Sai GS, Quqa Said, Garcia Cava D, Tcherniak Dmitri, Chatzi Eleni. A semi-supervised interpretable machine learning framework for sensor fault detection. *Smart Struct Syst Int J* 2021;29:251–66.
- [40] Manigrasso Francesco, Miro Filomeno Davide, Morra Lia, Lamberti Fabrizio. Faster-LTN: a neuro-symbolic, end-to-end object detection architecture. In: Artificial neural networks and machine learning–ICANN 2021: 30th international conference on artificial neural networks, Bratislava, Slovakia, September 14–17, 2021, proceedings, part II 30. Springer; 2021, p. 40–52.
- [41] Hochreiter Sepp, Schmidhuber Jürgen. Long short-term memory. *Neural Comput* 1997;9(8):1735–80.
- [42] Tay Yi, Bahri Dara, Metzler Donald, Juan Da-Cheng, Zhao Zhe, Zheng Che. Synthesizer: Rethinking self-attention for transformer models. In: International conference on machine learning. PMLR; 2021, p. 10183–92.
- [43] Pandya Yogesh. Gearbox fault diagnosis data. 2018, URL <https://data.openei.org/submissions/623>.
- [44] Cao Pei, Zhang Shengli, Tang Jiong. Gear fault data. 2018, <http://dx.doi.org/10.6084/m9.figshare.6127874.v1>, URL https://figshare.com/articles/dataset/Gear_Fault_Data/6127874.